

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA14

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO5: *Develop code optimization and Code Generation using DAG representation with data flow analysis to produce optimized target code. (BL3)*

Assessment Tool Weight-age: 40%

QUESTIONS: 5

Total Marks:50

Annexure A

SIMULATION TASK

Code of Conduct:

*I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Criteria	Weight age (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	20%	Demonstrates strong understanding of underlying concepts in the simulation	Shows good understanding with minor gaps	Partial understanding; some misconceptions	Lacks understanding of key concepts

Model Design	25%	Develops accurate, well-structured simulation/model independently	Develops correct model with minor errors	Model is incomplete or requires guidance	Unable to develop a proper model
Tool Usage	20%	Uses simulation tools effectively with correct setup and execution	Uses tools with minor errors or guidance	Limited ability to use tools properly	Unable to use simulation tools
Analysis of Results	20%	Provides clear, logical analysis and meaningful interpretation of results	Analysis is reasonable with minor gaps	Superficial analysis; limited interpretation	Unable to analyze or interpret results
Documentation	15%	Presents results clearly with proper documentation and structured reporting	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation

Annexure A

SIMULATION TASK

QUESTIONS

1. Given the three-address code sequence:

$t1 = x * 1$

$t2 = t1 + y$

$t3 = t2 - 0$

$t4 = t3 * 0$

if $t4 \neq 0$ goto L1

$t5 = y + y$

Simulate the **peephole optimization** process on this sequence. Produce the optimized instruction sequence and justify each optimization step applied, including algebraic simplification, constant folding, and dead code elimination.

2. Partition the following intermediate code into **basic blocks** and construct the corresponding **control flow graph**. Identify all leader statements and mention the rules used to identify them:

(1) $x = a + b$

(2) $y = x * c$

(3) if $y > 50$ goto 6

(4) $z = y - d$

(5) goto 7

(6) $z = y + d$

(7) $w = z * 2$

3. Construct the **DAG** for the following basic block:

$p = a + b$

$q = a + b$

$r = p * q$

$s = a + b$

$t = r + s$

Using the DAG, identify all **common sub-expressions**. Then simulate a simple code generator to produce optimized target code and explain the register allocation decisions at each step.

4. For a target machine with two registers **R0** and **R1**, simulate the working of a **simple code generator** for the code:

$t1 = a + b$

$t2 = c * d$

$t3 = t1 - t2$

$t4 = e + f$

$t5 = t3 / t4$

Show the **register descriptor** and **address descriptor** after each instruction is processed.

5. For a **RISC-style target machine** with instructions **LOAD**, **STORE**, **ADD**, **SUB**, **MUL**, **MOV**, simulate target code generation for the expression:

$(a - b) * (c + d) - e$

Show the complete sequence of target instructions generated. Also explain the **runtime storage management** and **stack frame layout** used during the computation.

Given the three-address code sequence:

```
t1 = a + b,  
t2 = t1 + 0,  
t3 = t2 * 1,  
if t3 == 0 goto L1,  
t4 = t3 + t3
```

ANSWER

Q1. Peephole Optimization

Given:

```
t1 = x * 1  
t2 = t1 + y  
t3 = t2 - 0  
t4 = t3 * 0  
if t4 != 0 goto L1  
t5 = y + y
```

Step-by-step optimization

Step	Original	Optimization	Result
1	t1 = x * 1	$x * 1 = x$	t1 = x
2	t2 = t1 + y	Replace t1 with x	t2 = x + y
3	t3 = t2 - 0	$a - 0 = a$	t3 = t2
4	t4 = t3 * 0	$a * 0 = 0$	t4 = 0
5	if t4 != 0 goto L1	0 != 0 is always false	Remove branch
6	t5 = y + y	$y + y = 2*y$	t5 = 2*y

Since t1, t3, and t4 are no longer required after propagation, they can be eliminated.

Optimized sequence

$t2 = x + y$

$t5 = 2 * y$

Optimizations used:

- Algebraic simplification: $x * 1 \rightarrow x$, $t2 - 0 \rightarrow t2$
- Constant folding: $t3 * 0 \rightarrow 0$, $0 != 0 \rightarrow \text{false}$
- Dead-code elimination: removes unused temporaries and unreachable branch

Q2. Basic Blocks and Control Flow Graph

Given:

(1) $x = a + b$

(2) $y = x * c$

(3) if $y > 50$ goto 6

(4) $z = y - d$

(5) goto 7

(6) $z = y + d$

(7) $w = z * 2$

Leader identification rules

A statement is a **leader** if:

1. The first statement is always a leader.
2. The target of a conditional/unconditional jump is a leader.
3. The statement immediately following a conditional/unconditional jump is a leader.

Leaders

- (1) $\rightarrow$ first statement

- (4) → follows conditional jump at (3)
- (6) → target of goto 6
- (7) → target of goto 7 and follows goto 7

Therefore:

Leaders = (1), (4), (6), (7)

Basic Blocks

B1:

- (1) $x = a + b$
- (2) $y = x * c$
- (3) if $y > 50$ goto 6

B2:

- (4) $z = y - d$
- (5) goto 7

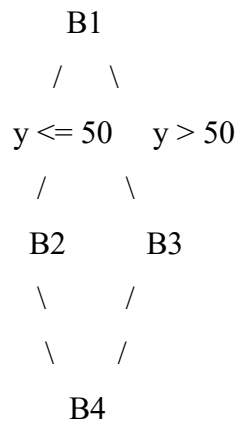
B3:

- (6) $z = y + d$

B4:

- (7) $w = z * 2$

Control Flow Graph



Edges:

$B1 \rightarrow B2$

$B1 \rightarrow B3$

$B2 \rightarrow B4$

$B3 \rightarrow B4$

Q3. DAG Construction and Common Sub-expressions

Given:

$p = a + b$

$q = a + b$

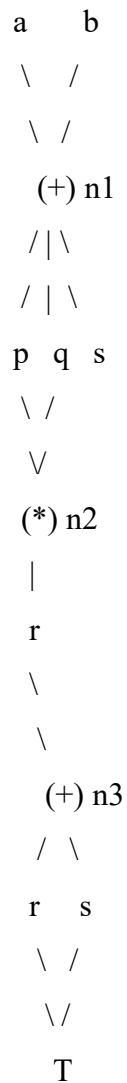
$r = p * q$

$s = a + b$

$t = r + s$

DAG

The expression $a+b$ occurs three times, so only **one DAG node** is needed for it.



Common sub-expression

$a + b$

occurs in:

$p = a + b$

$q = a + b$

$s = a + b$

So it should be computed only once.

Optimized three-address code

$t1 = a + b$

$t2 = t1 * t1$

$t3 = t2 + t1$

Then:

$p = t1$

$q = t1$

$r = t2$

$s = t1$

$t = t3$

Simple target code

Assuming registers R0 and R1:

LOAD R0, a

ADD R0, b ; $R0 = a+b$

MOV R1, R0

MUL R1, R0 ; $R1 = (a+b)*(a+b)$

ADD R1, R0 ; $R1 = r+s$

STORE t, R1

The important register decision is to **keep $a+b$ in R0**, because it is needed three times.

Q4. Code Generation Using R0 and R1

Given:

$t1 = a + b$

$t2 = c * d$

$t3 = t1 - t2$

$t4 = e + f$

$t5 = t3 / t4$

Assume:

- R0 and R1 are available.
- Register descriptor tells what each register contains.
- Address descriptor tells where each temporary/value is located.

Step 1: $t1 = a + b$

ADD R0, a, b

Descriptors:

Register	Contents
R0	$t1 = a+b$
R1	—

Address descriptor:

Value	Location
a	memory
b	memory
t1	R0

Step 2: $t2 = c * d$

MUL R1, c, d

Register	Contents
R0	t1
R1	t2

Address descriptor:

$t1 \rightarrow R0$

$t2 \rightarrow R1$

Step 3: $t3 = t1 - t2$

SUB R0, R1

Now R0 contains t3.

Register	Contents
R0	$t3 = t1 - t2$
R1	t2

Address descriptor:

$t3 \rightarrow R0$

$t2 \rightarrow R1$

R1 can now be reused because t2 is no longer needed.

Step 4: $t4 = e + f$

ADD R1, e, f

Register	Contents
R0	t3
R1	t4

Address descriptor:

$t3 \rightarrow R0$

$t4 \rightarrow R1$

Step 5: $t5 = t3 / t4$

DIV R0, R1

Final state:

Register	Contents
R0	$t5 = t3/t4$
R1	$t4$

Address descriptor:

$t5 \rightarrow R0$

$t4 \rightarrow R1$

Final target code

ADD R0, a, b

MUL R1, c, d

SUB R0, R1

ADD R1, e, f

DIV R0, R1

The main strategy is to **reuse R0 for the result being carried forward** and reuse R1 whenever its previous value is dead.

Q5. RISC Target Code Generation

Expression:

$$(a - b) * (c + d) - e$$

Evaluation order

First calculate:

$$T1 = a - b$$

$$T2 = c + d$$

$$T3 = T1 * T2$$

$$\text{Result} = T3 - e$$

RISC-style target code

Assuming R0, R1, and R2:

LOAD R0, a ; R0 = a

LOAD R1, b ; R1 = b

SUB R0, R0, R1 ; R0 = a-b

LOAD R1, c ; R1 = c

LOAD R2, d ; R2 = d

ADD R1, R1, R2 ; R1 = c+d

MUL R0, R0, R1 ; R0 = (a-b)*(c+d)

LOAD R1, e ; R1 = e

SUB R0, R0, R1 ; R0 = (a-b)*(c+d)-e

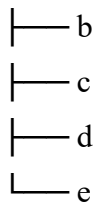
Final result is in **R0**.

Runtime storage management

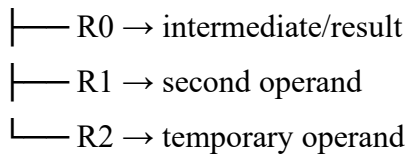
During execution:

Memory

|— a



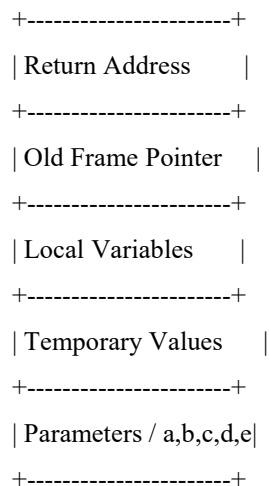
Registers



The compiler tries to keep intermediate results in registers to reduce memory accesses.

Conceptual stack-frame layout

Higher Address



Lower Address

The stack frame is created when the function starts and removed when it returns. In this expression, registers are sufficient for the intermediates, so **no temporary needs to be spilled to the stack**.

Additional Peephole Optimization

For your final sequence:

```
t1 = a + b
t2 = t1 + 0
t3 = t2 * 1
if t3 == 0 goto L1
t4 = t3 + t3
```

Step 1: Algebraic simplification

```
t2 = t1 + 0
```

becomes:

```
t2 = t1
```

Then:

```
t3 = t2 * 1
```

becomes:

```
t3 = t2
```

Therefore:

```
t3 = t1
```

Step 2: Copy propagation

Replace t3 by t1:

```
if t1 == 0 goto L1
t4 = t1 + t1
```

Optimized code

```
t1 = a + b
if t1 == 0 goto L1
t4 = t1 + t1
```

Or, using strength reduction:

```
t1 = a + b
```

if t1 == 0 goto L1

t4 = 2 * t1

Optimizations applied: algebraic simplification, copy propagation, redundant temporary elimination, and strength reduction.

RUBRICS FOR CALCULATION

Simulation tools

Criteria	Weight age (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	20%	Demonstrates strong understanding of underlying concepts in the simulation	Shows good understanding with minor gaps	Partial understanding; some misconceptions	Lacks understanding of key concepts
Model Design	25%	Develops accurate, well-structured simulation/model independently	Develops correct model with minor errors	Model is incomplete or requires guidance	Unable to develop a proper model
Tool Usage	20%	Uses simulation tools effectively with correct setup and execution	Uses tools with minor errors or guidance	Limited ability to use tools properly	Unable to use simulation tools
Analysis of Results	20%	Provides clear, logical analysis and meaningful interpretation of results	Analysis is reasonable with minor gaps	Superficial analysis; limited interpretation	Unable to analyze or interpret results
Documentation	15%	Presents results clearly with proper documentation and structured reporting	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation